

Maze Escape: Human vs Monster AI Survival Game

Group Number: XX

Member 1	Member 2	Member 3
Member 4	Member 5	Member 6

June 2026

Course Code: SOF106

Course Name: Principles of Artificial Intelligence

Academic Session: 202604

Assessment Title: Final Assessment Group Project

Contents

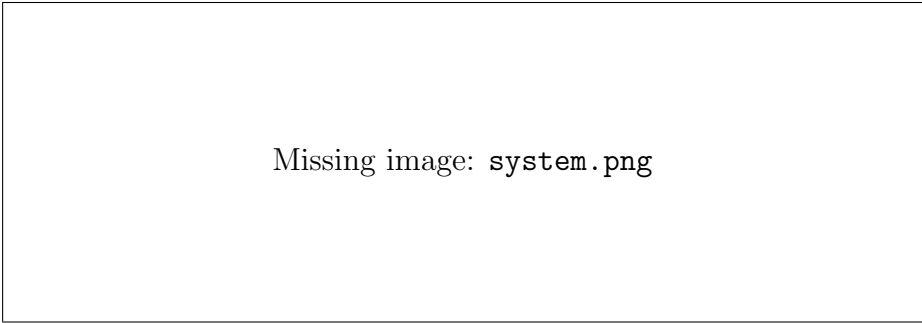
1	Introduction	3
2	Project Objectives	4
3	Methodology	4
3.1	Map Representation	4
3.2	Genetic Algorithm for Map Generation	5
3.3	Human AI Algorithm (BFS)	5
3.4	Monster AI Algorithms	5
3.4.1	Greedy Search	5
3.4.2	Minimax	5
3.4.3	Simulated Annealing	5
3.4.4	A* Algorithm	5
3.4.5	A* for Double Monsters	6
3.5	Item System	6
4	Implementation	6
5	Validation and Verification	7
5.1	Map Generation Test	7
5.2	Spawn Test	7
5.3	Boundary Test	7
5.4	Algorithm Comparison	7
6	Results and Discussion	7
7	Conclusion	7

Abstract

Our project is a survival game called **Maze Escape**. The goal of the project is to show the practical use of artificial intelligence search and optimization algorithms. The game simulates humans being chased by monsters on a 30×30 circular grid map (wrapping edges). The system uses and compares five AI search strategies: **A* pathfinding**, **Greedy Algorithm**, **breadth-first search (BFS)**, **Minimax Algorithm**, and **Simulated Annealing Algorithm**. The maze is generated by a **Genetic Algorithm (GA)**. The GA improves map connectivity, wall density, path diversity, and branch structure through multi-generation evolution.

This project shows how artificial intelligence behaves under limited, competitive, and real-time conditions. It also shows how different search strategies change the difficulty of the game and affect the player experience. The A* algorithm creates the most direct pursuit path. The greedy algorithm gives fast performance with low computing cost. The simulated annealing algorithm adds controlled randomness and makes monster movement less predictable. The genetic algorithm keeps generating connected and navigable maps to maintain gameplay quality.

1 Introduction



Missing image: `system.png`

Figure 1: Game System

In the course “Principles of Artificial Intelligence”, we learned a variety of search, planning and optimization algorithms. Based on these core algorithms, our maze escape project improves static single-intelligent path search to dynamic multi-dimensional confrontation scenarios: in order to achieve this transformation, we introduce randomly generated maze maps based on genetic algorithms, expand dimensional space and create two-way tracking mechanisms.

In order to comprehensively evaluate these algorithms, we have designed two complementary game modes: escape mode and chase mode. In the chase mode, the actions of human characters are driven by circumvention algorithms based on breadth priority search (BFS); and for the core task of chasing predators and prey, the project adopts four different intelligent strategies:

- **A* Search Algorithm:** In single-monster mode, the algorithm calculates the shortest path to the player. In dual-monster mode, monsters are divided into two types: direct pursuers and interceptors; the interceptors predict the player’s next move and block escape routes.

- **Greedy Algorithm:** It calculates a distance field centered at the player’s current position, causing monsters to always move toward the nearest neighboring node. This pursuit method is highly efficient and requires low computational resources.
- **Minimax Decision Tree:** treats the pursuit process as a two-player game. The algorithm combines α - β pruning with a substitution table to predict subsequent moves. The monster aims to reduce the player’s safety score during the game, while simultaneously simulating the player’s efforts to improve it, enabling efficient pursuit from both perspectives.
- **Simulated Annealing Algorithm:** The algorithm first generates an initial movement path and then randomly adjusts it through repeated iterations. Following the cooling strategy, the system accepts suboptimal moves with a certain probability, making the monster’s actions more random and harder to predict.

The central environment of our project is a 30×30 toroidal grid map, where the maze layouts are generated by a highly customized Genetic Algorithm (GA). The algorithm keeps a population of candidate maps and evolves them for more than 20 generations. The fitness function balances several important factors, including strong floor connectivity, about 28% wall density, a high number of three-way and four-way junctions, and an average BFS path length close to 20 steps between randomly selected positions. The system also uses tournament selection, row and column crossover, and bit-flip mutation. These methods help the algorithm generate maps with good accessibility, spatial diversity, and suitable difficulty.

The following report will introduce the design and specific implementation methods of each component in detail, quantitatively analyze the algorithm characteristics of each artificial intelligence strategy and its impact on the gameplay, and reflect on the respective advantages and limitations observed in the development process.

2 Project Objectives

The main objectives of this project are:

- To develop a playable human versus monster survival game.
- To apply artificial intelligence algorithms in a game environment.
- To generate playable maps using a Genetic Algorithm.
- To implement different monster AI strategies.
- To compare the behavior and performance of different algorithms.

3 Methodology

3.1 Map Representation

The game map is represented as a 30 by 30 two-dimensional grid. In the map, 0 represents a walkable floor cell, while 1 represents a wall. The human and monsters can only move on walkable cells.

The map also uses a wrap-around boundary system. For example, if a character moves left from column 0, it will appear at column 29. This is implemented by using the modulo operation.

3.2 Genetic Algorithm for Map Generation

A Genetic Algorithm is used to generate playable maps. Each map is treated as one chromosome. A chromosome is represented as a 30 by 30 binary grid, where 0 means floor and 1 means wall.

The initial population contains several randomly generated maps. Each map is evaluated using a fitness function. The fitness function considers reachability, wall ratio, junction complexity, and path distance. Reachability checks whether most walkable cells are connected. Wall ratio prevents the map from being too empty or too blocked. Junction complexity encourages more branching paths. Path distance helps keep the chase distance reasonable.

After fitness evaluation, tournament selection is used to choose better maps as parents. Crossover combines two parent maps by cutting and joining their rows or columns. Mutation randomly changes a small number of cells from floor to wall or from wall to floor. After several generations, the best map is saved as a text file using 0 and 1 values separated by spaces.

3.3 Human AI Algorithm (BFS)

When the user plays the role of monster, a Breadth-First Search(BFS)-based system is used to control the human character. Generally, the objective of the human character is to keep as far away from the monster as possible. Therefore, every time when the Human AI needs to decide its move direction, we run BFS starting from the monster's current position. This creates a distance map over the whole maze, where each reachable cell stores how many steps the monster would need to reach it. Since the maze is an unweighted grid, BFS gives the shortest path distance correctly.

After building this distance map, the Human AI checks the four possible next positions of the human: up, down, left, and right. It chooses the move with the largest BFS distance from the monster, meaning the human tries to move to the neighboring cell that is farthest away in terms of actual maze path distance.

3.4 Monster AI Algorithms

Several AI algorithms are used for monster movement.

3.4.1 Greedy Search

3.4.2 Minimax

3.4.3 Simulated Annealing

3.4.4 A* Algorithm

A* is a heuristic graph search algorithm. It finds the shortest path between two nodes by adding the cost of the lined path to the residual cost estimate of reaching the target node. The cost of each node is calculated by the following function:

$$f(n) = g(n) + h(n)$$

In the process of the monster moving to the player in each round, the map grid is regarded as an untitled map, thus calculating the path cost. Due to randomly generated roadblocks, each walkable grid cell has up to four neighbors, which can be reached by moving up, down, left or right. When calculating heuristic functions, we use the Manhattan distance between the current cell and the player's location. In addition, because the map has a ring structure, we will apply modular operations to the row index and column index every time we expand the neighbor. This allows movement to cross the grid boundary. After the algorithm is executed, the core function `astar(grid, start, goal)` will return the full path from the current position of the monster to the player. With each move of the player, the Astar algorithm will calculate the corresponding optimal path. If there is no path (for example, the player is completely surrounded by a wall), the monster will stay in place.

3.4.5 A* for Double Monsters

In the double monster A* mode, in order to improve the probability of the player being caught up, the algorithm will give each monster a different role, and also ensure that the two monsters cannot enter the same path. Among them, the monster closest to the player (measured by Manhattan distance) is the chaser, which directly uses the A* algorithm to chase the player. The monster far away from the player is the interceptor. It predicts the future direction of the player and moves to the interception point, so as to achieve enveloping.

For the character setting of the second monster, the method of determining the interception point is to project the player's position forward, point from the chaser to the player's direction, and fix the offset of the four grid units. The offset will be adjusted according to the ring topology. If the projection cell is a wall, the algorithm will consider whether it can penetrate the wall. If not, the interception target will return to the player's current position. In this case, the interceptor will also chase the player directly. This kind of character allocation means that two monsters will appear from different angles, which will make it more difficult for players to escape. The key functions of this pattern are `get_next_step_single` and `get_next_steps_two`. `get_next_steps_single` is used to return the next cell chased by a single monster, while `get_next_steps_two` is used to set the role of each monster before calculating the next step of each monster using the A* algorithm.

3.5 Item System

Items are generated after a fixed number of human steps. For example, new items may appear every 10 human steps. Unused items remain on the map. Items can affect the movement and strategy of the game, such as giving the human a speed boost or temporarily stopping the monster.

4 Implementation

The project is divided into several modules. The AI folder stores the algorithm implementations, including Genetic Algorithm, BFS, Greedy Search, A*, Minimax, and Simulated

Annealing. The game folder stores the main game logic, including map handling, player movement, monster movement, item logic, collision checking, and scoring.

The Genetic Algorithm module outputs a text file containing the generated map. The game system can then load this map and use it during gameplay.

5 Validation and Verification

Several tests are used to validate the project.

5.1 Map Generation Test

The generated map should have 30 rows and 30 columns. Each cell should only contain 0 or 1. The wall ratio should be reasonable, and most walkable cells should be connected.

5.2 Spawn Test

The human and monster should spawn on walkable cells. They should not spawn on the same position. Their starting distance should also be within a reasonable range to avoid unfair gameplay.

5.3 Boundary Test

The wrap-around boundary is tested by moving characters across the edges of the map. Moving left from column 0 should move the character to column 29. Moving up from row 0 should move the character to row 29.

5.4 Algorithm Comparison

The monster AI algorithms can be compared using capture steps, runtime, and behavior. Greedy Search is simple and fast. A* is more stable for pathfinding. Minimax can predict future movement but requires more computation. Simulated Annealing provides more random behavior.

6 Results and Discussion

The project successfully generates playable maps and supports AI-based monster chasing behavior. The Genetic Algorithm improves map quality compared with purely random map generation. BFS can support human escape planning. A* provides more reliable chasing behavior than Greedy Search when obstacles exist.

The project also shows that different algorithms have different strengths and weaknesses. Simple algorithms are easier to run but may perform poorly in complex maps. More advanced algorithms can make better decisions but may require more computation.

7 Conclusion

In conclusion, this project demonstrates how artificial intelligence algorithms can be applied in an interactive survival chase game. The Genetic Algorithm is used for map

generation, BFS is used for human path searching, and several monster AI algorithms are used for chasing behavior. Through this project, the team gained practical experience in AI search algorithms, optimization, game logic, and system integration.

Future improvements may include better multi-agent monster cooperation, more advanced item effects, improved user interface design, and reinforcement learning-based monster behavior.